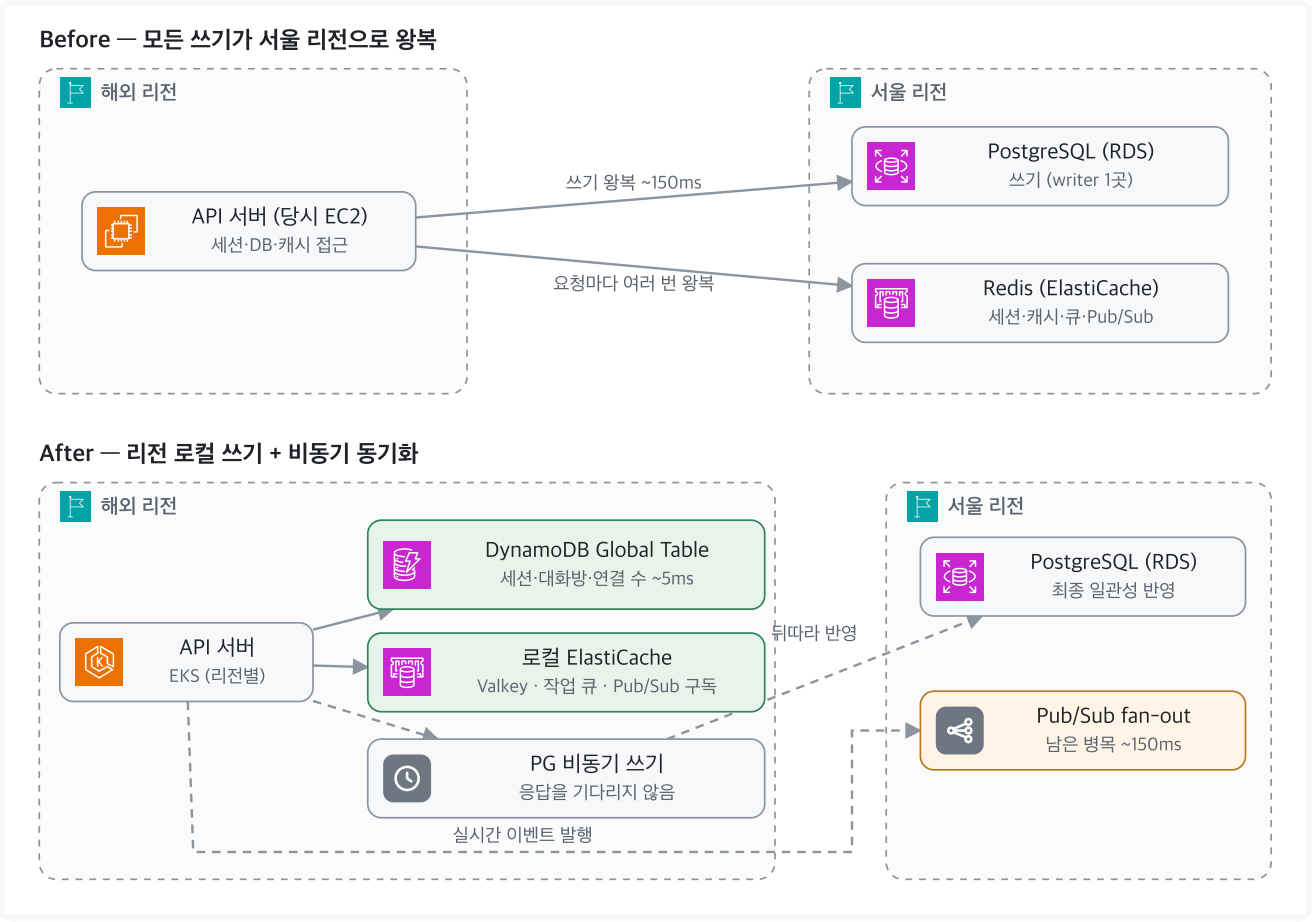
경력기술서

플리토 · 실시간 AI 통번역

글로벌 리전 레이턴시 개선

2025.11 – 2026.05 | 설계 · 구현 주도 | DynamoDB · Redis · PostgreSQL | 재현 실험 코드 →



문제 해외 리전에서 쓰면 국내보다 4~5배 느리다는 피드백. 유력한 원인은 서울 리전에 묶인 DB·Redis 쓰기 왕복(1회 약 150ms)이 요청마다 여러 번 발생하는 구조.

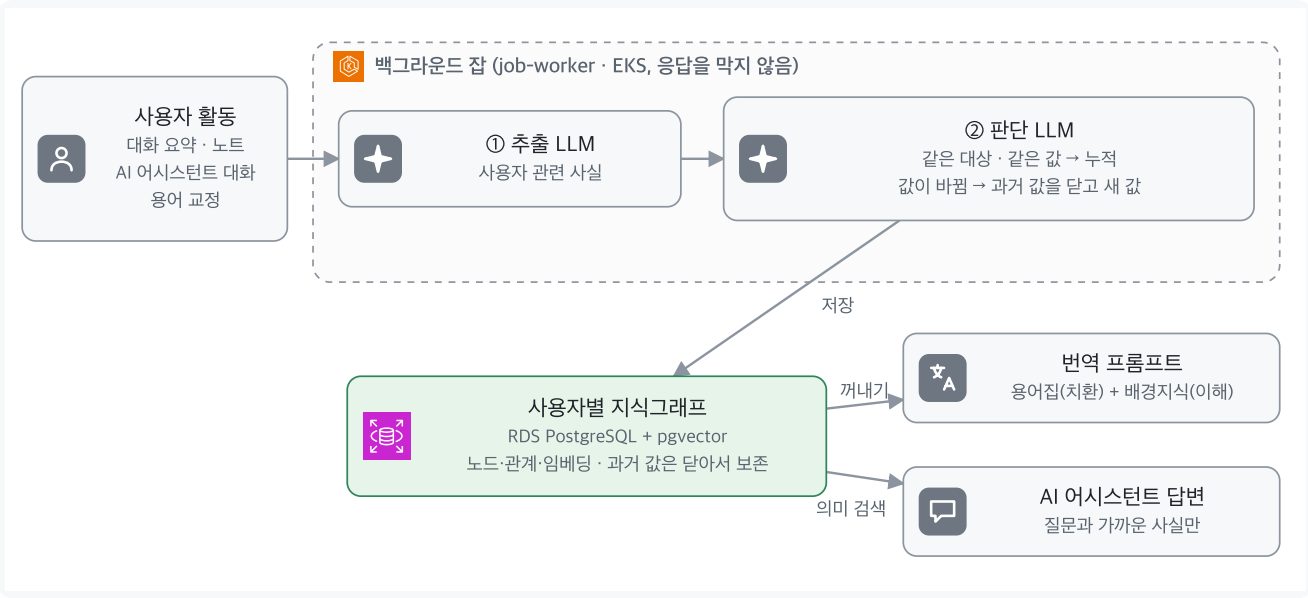
판단 Aurora Global Database·Aurora DSQL 검토 — 쓰기는 여전히 한 리전이거나 대륙 간 미지원이고 비용도 더 높아 보류. 실시간성이 강한 데이터만 리전 로컬 쓰기로 옮기기로 결정.

해결 대화방·세션·참가자 연결 수를 DynamoDB Global Tables로, 작업 큐를 리전별 Redis로 옮기고, 실시간 메시지는 모든 리전에 발행해 각 리전에서 구독. PostgreSQL은 응답을 기다리지 않고 뒤따라 쓰게 하고(DynamoDB 실패 시 PostgreSQL로 처리), 기능 플래그로 언제든 기존 구조로 되돌릴 수 있게 함.

결과 Close API **1,673 → 327ms (-80%)**, Create API **2,630 → 90ms (-97%)** — 해외 상황을 재현한 개발 환경에서 15회 측정. 남은 병목은 Pub/Sub fan-out.

지식그래프 기반 초개인화

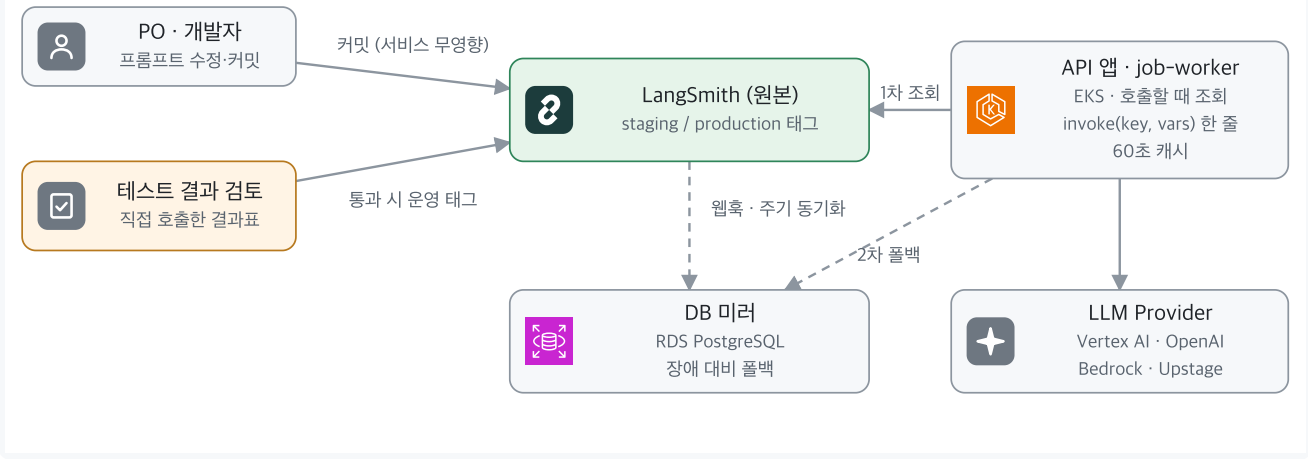
2026.01 – 2026.10 | 백엔드 설계 · 구현 · 상용 반영 | PostgreSQL · pgvector · LLM



문제	사용자가 여러 번 고친 표기나 자신의 분야 맥락을 번역·AI 답변이 기억하지 못함.
해결	사용자 활동에서 LLM이 사실을 추출하고, 기존 사실과 같은 대상인지·값이 바뀌었는지 LLM이 다시 판단해 같으면 누적, 바뀌면 과거 값을 닫고 새 값으로 사용자별 그래프에 저장. 번역에는 용어집·배경지식으로, AI 답변에는 질문과 의미가 가까운 사실만 넣음. 모든 쌓기는 백그라운드 잡.
개선	상용 데이터를 보니 노트 발표자의 이야기가 사용자 사실로 섞이는 문제가 있어 "사용자 중심" 관계 구조로 재설계. 고정 기준선 게이트로 회귀 없이 채택한 뒤 상용 사용자 이관 완료.

LLM 프롬프트 관리 체계

2026.06 | 설계 · 구현 | LangSmith



문제	프롬프트가 코드에 있어 문장 하나를 고쳐도 배포가 필요했고, 바뀐 버전의 품질을 확인할 방법이 없음.
해결	LangSmith를 원본으로 두고 서버가 직접 조회(60초 캐시, 장애 시 DB 미러 폴백). 개발/운영은 태그로 분리하고, 프롬프트를 직접 호출한 테스트 결과표를 검토해 통과한 버전만 운영 태그를 받게 함.
결과	코드 배포 없이 프롬프트를 고쳐 약 1분 안에 반영 — PO·PM용 사용 가이드를 함께 제공.

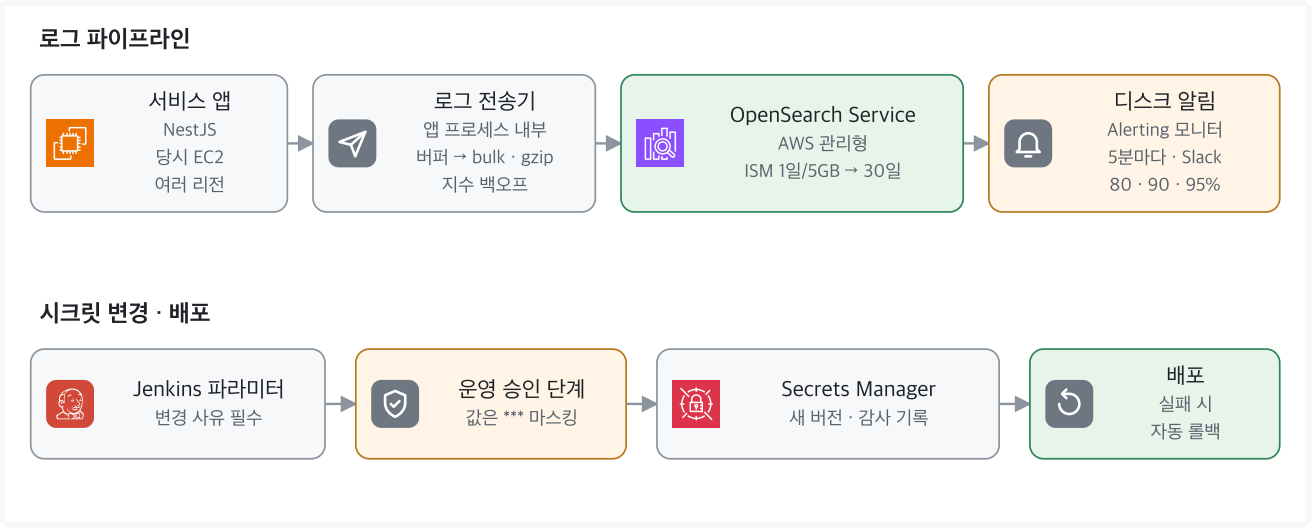
외부 AI 에이전트 연결(MCP) — 기능 제안

2026.10 – 검토 중 · PRD · Figma · 서버 명세

목표	고객이 쓰는 AI 에이전트(Claude · ChatGPT · Cursor 등)가 대화·노트 기록을 읽고 용어집을 미리 준비하게 함.
한 일	타사 연결 방식·사용 한도를 비교 조사해 PRD를 쓰고, Figma 시안(데스크톱·모바일 웹)·서버 명세·아키텍처 그림을 한 벌로 작성. 이 문서 방식을 이후 기능(음성 파일 업로드)에도 적용 중.
상태	문서 검토 후 구현 예정.

로그 · 배포 운영 안정화

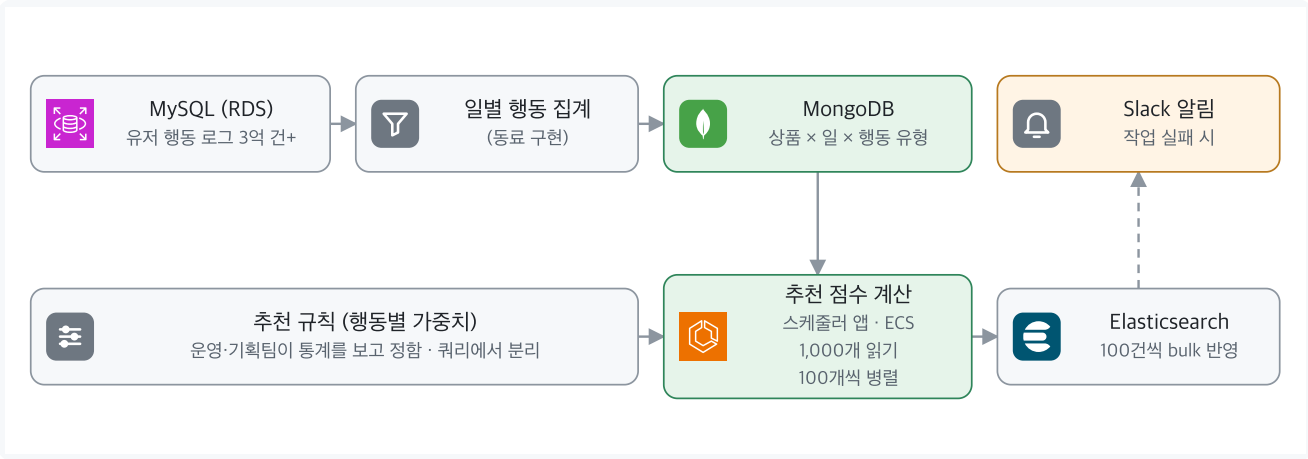
2025.10 – 2026.05 · 구축 · 장애 복구 · OpenSearch · Jenkins



로그	여러 리전의 서비스 로그를 OpenSearch로 보내는 앱 내부 전송기를 동료와 함께 구축(버퍼에 모아 bulk 전송 · 전송 실패가 서버에 영향 없게). 이후 디스크 풀 장애는 debug 로그 유입이 원인이라 유입을 막고, 보관 정책·압축·샤드 설정을 기본값으로 고정한 뒤 OpenSearch Alerting으로 디스크 80 · 90 · 95% Slack 알림과 대응 절차를 만듦. 수집은 k8s 전환 후 2026.07부터 노드별 fluent-bit으로 바뀜.
배포	시크릿 변경을 Jenkins 파이프라인으로 일원화(변경 사유 필수 · 값 마스킹 · 운영 승인). 서버 CPU 포화 장애 뒤 불필요한 로그를 정리하고, 배포가 실패하면 이전 버전으로 되돌리는 배포·로그 보관 정책을 도입.

추천 상품 스케줄러 재구축

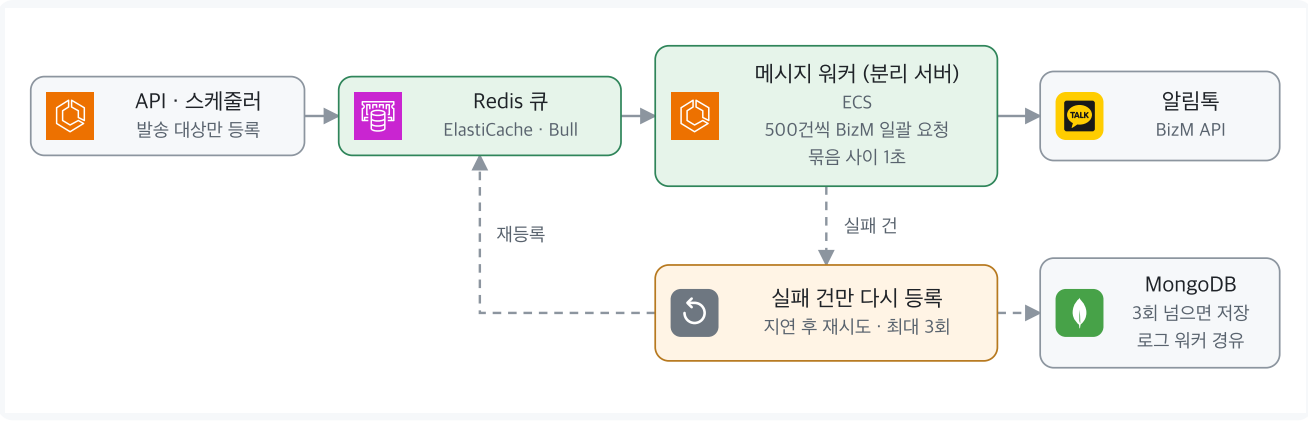
MySQL · MongoDB · Elasticsearch



방식	개인화 모델이 아니라, 브랜딩·운영팀과 기획자가 통계 데이터를 보고 정한 규칙(행동별 가중치)으로 상품 점수를 매겨 추천 순서를 내려주는 룰 기반 추천.
문제	상품마다 3억 건 이상의 행동 로그에서 2주치 클릭·좋아요·장바구니 수를 세는 서브쿼리를 붙이고 주문·리뷰까지 함께 조회해 실행에 60분 이상 걸림. 가중치(클릭 5 · 좋아요 10 · 장바구니 20)도 쿼리에 고정돼 3년 넘게 손대지 못함.
해결	원본 로그 대신 동료가 만든 상품별·일별 행동 집계(MongoDB)를 읽도록 점수 계산을 다시 작성. 상품은 1,000개씩 읽고 점수는 100개씩 묶어 병렬로 계산한 뒤 Elasticsearch에 100건씩 bulk로 반영. 가중치는 쿼리에서 빼 설정값으로 분리하고, 작업이 실패하면 Slack으로 알림.
결과	스케줄러 실행 시간 **60분 → 15분**.

알림 메시지 워커

설계 · 구현 | Bull(Redis) · ECS · MongoDB



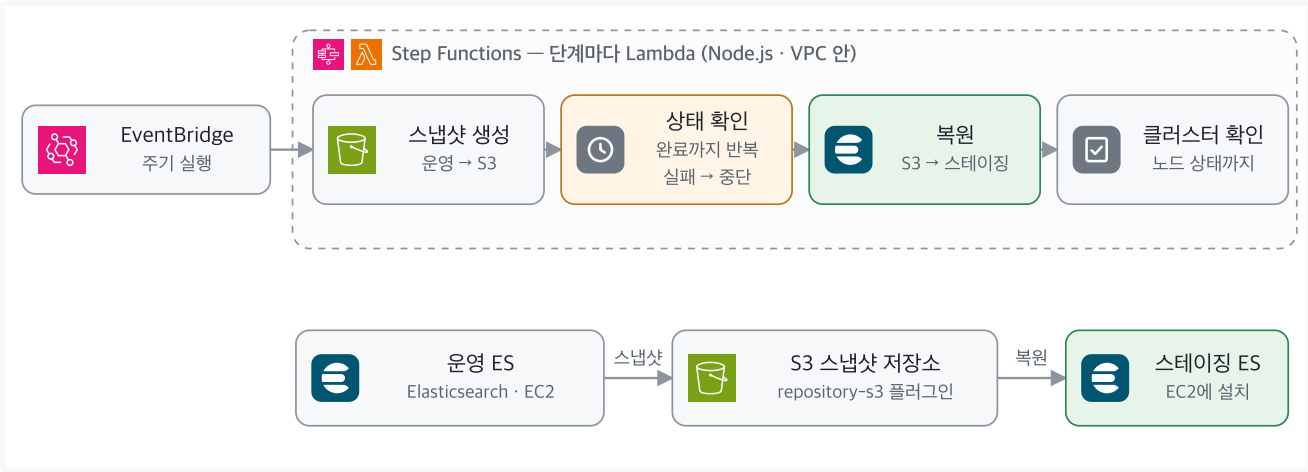
문제 쿠폰 만료 안내 같은 알림톡을 약 100만 명에게 주기적으로 보내는 작업이 API 서버에 큰 부담을 줌. 발송에 실패한 메시지를 다시 보내거나 기록하는 체계도 없었음.

해결 발송을 별도 워커 서버(ECS)로 분리. API·스케줄러는 발송 대상만 Redis 큐(Bull)에 넣고, 워커가 500건씩 묶어 BizM에 일괄 요청하며 묶음 사이에 1초를 둬 외부 API에 요청이 몰리지 않게 함. 응답에서 실패한 건만 모아 시도 횟수를 붙여 지연 후 큐에 다시 넣고(성공한 건은 다시 보내지 않음), 3회를 넘으면 로그 워커 큐를 거쳐 MongoDB에 실패 이력을 남김.

판단 다른 워커에서 이미 쓰던 Bull 큐를 그대로 써서 새 인프라 없이 분리. 알림톡·SMS·이메일 중 무엇으로 보낼지는 도메인별 잡이 정하게 해, 새 알림은 잡 하나로 추가할 수 있게 함.

Elasticsearch 스냅샷 마이그레이션 자동화

설계 · 구현 | Step Functions · Lambda · EventBridge · S3



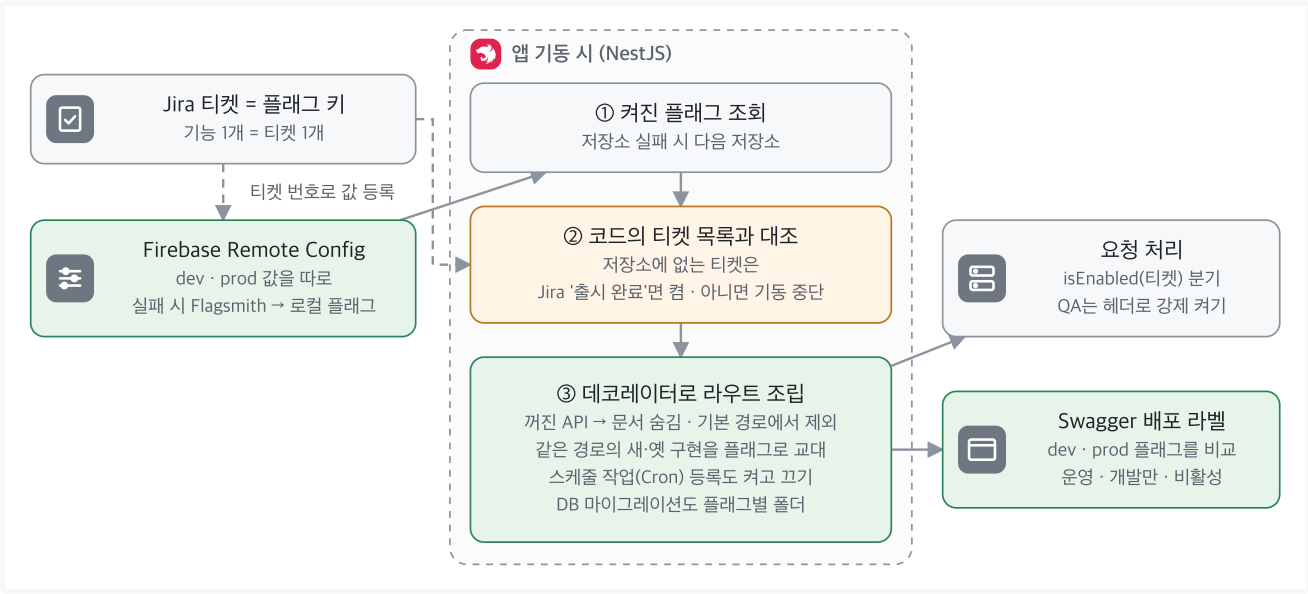
문제 스테이징 검색 데이터를 운영과 맞추려면 스냅샷 생성 → 완료 확인 → 복원을 사람이 순서대로 해야 했고, 중간에 실패해도 알아채기 어려움.

해결 EC2에 설치된 Elasticsearch에 repository-s3 플러그인과 IAM 역할을 설정해 S3를 스냅샷 저장소로 연결. 스냅샷 생성 → 완료될 때까지 상태 확인(실패면 중단) → 스테이징 복원 → 클러스터·노드 상태 확인을 Step Functions 단계로 묶고, 각 단계는 VPC 안에서 ES를 호출하는 Node.js Lambda로 작성. EventBridge로 주기 실행.

판단 스냅샷은 데이터 크기에 따라 오래 걸려 Lambda 한 번(최대 15분) 안에서 기다리기 어려움 → 요청과 확인을 짧은 단계로 나누고, 순서·대기·실패 분기는 Step Functions가 맡게 함.

Feature Flag 기반 Trunk-Based 개발

팀 도입 · Swagger 배포 라벨 구현 | Firebase Remote Config · NestJS · GitHub Actions



문제 dev · release 브랜치를 따로 관리해 공통 코드 작업이 자주 충돌하고, 운영 배포가 늦어지면 개발 환경과 내용이 어긋남. 공통 모듈을 고쳐도 운영 배포 전까지는 다른 작업에서 쓰기 어려움.

방식 팀에서 master 하나로 개발하고 기능은 플래그로 켜고 끄는 방식을 도입. 플래그 키는 Jira 티켓 번호, 값은 Firebase Remote Config에 개발 · 운영을 따로 둠(조회 실패 시 Flagsmith → 로컬 플래그). 앱이 뜰 때 켜진 플래그를 읽어 라우트를 조립 — 꺼진 API는 문서에서 숨기고 기본 경로에서 빼며, 같은 경로의 새 · 옛 구현을 플래그로 교대하고, 스케줄 작업과 DB 마이그레이션도 플래그별로 켬. 코드에는 있는데 저장소에 없는 티켓은 Jira에서 '출시 완료'인 것만 켜고 아니면 기동을 멈춰 설정 누락을 막음. master에 병합하면 GitHub Actions가 dev · release PR을 자동 생성하고, 출시된 기능은 일정 기간 뒤 플래그와 옛 코드를 지움.

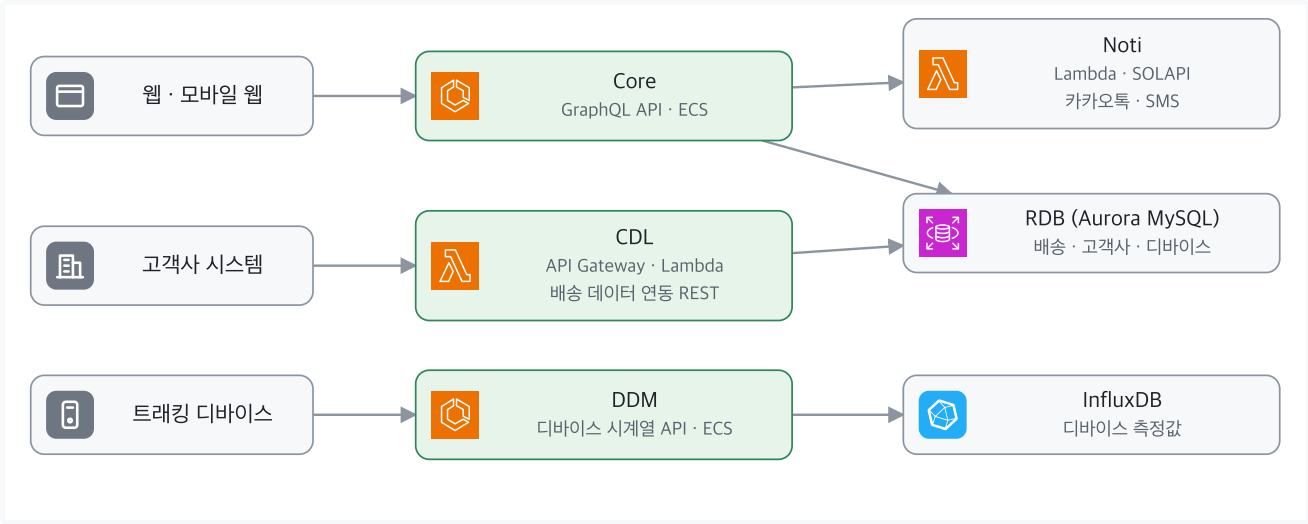
내 구현 동료가 만든 플래그 모듈 위에, 개발 · 운영 플래그를 비교해 Swagger에 API별 배포 상태 라벨(운영 · 개발만 · 비활성)을 자동으로 붙이는 기능 — 클라이언트 개발자가 오래 요청하던 "이 API가 어디까지 배포됐나"를 문서에서 바로 확인.

AWS 권한 정책

모든 권한을 가진 단일 그룹 → 신규 입사자는 읽기 전용, 업무별 권한 그룹(EC2 · RDS · ElastiCache · CI/CD 등), 시크릿 · 배포 같은 민감 작업은 별도 그룹을 상위 관리자 승인으로 부여하는 최소 권한 구조로 재설계.

산업군 통합 관제 플랫폼 — 기능 기준 서버 분리

2022.12 – 2023.09 · 백엔드 설계 · 구현 · NestJS · GraphQL · ECS · Lambda · InfluxDB



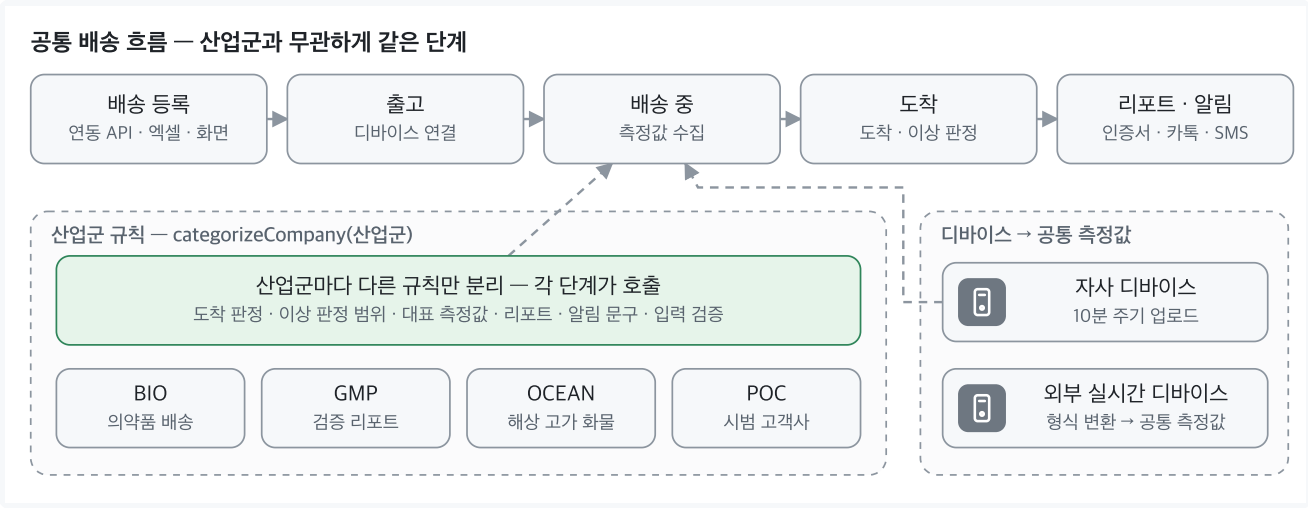
문제 의약품 배송 모니터링에서 시작해 산업군마다 서비스를 따로 만들다 보니 관리할 서버와 엔드포인트가 계속 늘어날 상황. 레거시에서는 고객사 연동 API 트래픽이 몰리면 서비스 전체가 영향을 받은 적이 있음.

해결 배송 도메인을 하나로 합친 플랫폼을 설계하는 데 참여해 백엔드를 맡음. 부하 성격이 다른 기능끼리 서버를 나눔 — 화면용 GraphQL API(Core), 디바이스 시계열(DDM), 고객사 연동 REST(CDL), 카카오톡 · SMS 알림(Noti).

판단 상시 트래픽을 받는 Core · DDM은 ECS, 요청이 몰렸다 사라지는 연동 · 알림은 Lambda로 둬 연동 폭주가 화면 API에 번지지 않게 하고, 쓰는 만큼만 비용을 냄.

공통 배송 흐름 + 산업군 규칙

의약품 · GMP 검증 · 해상 고가 화물 | 디바이스 호환



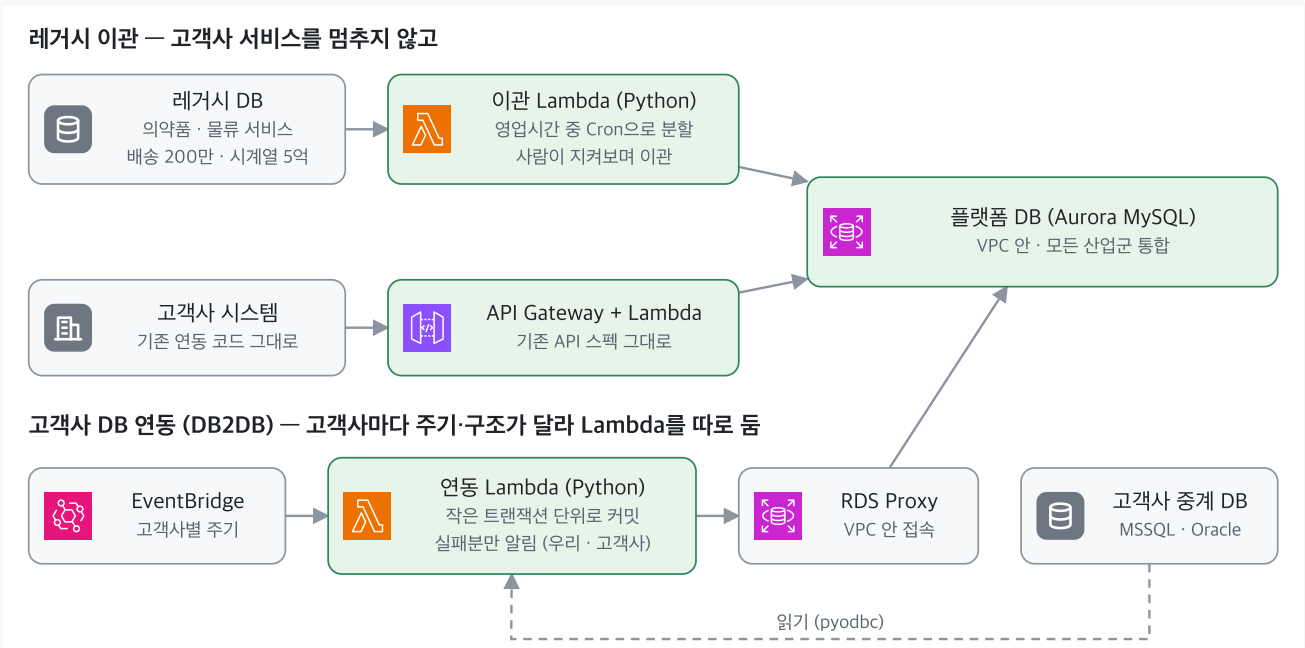
문제 의약품 · GMP 검증 · 해상 고가 화물은 도착을 판정하는 기준, 이상으로 보는 온도 · 습도 범위, 리포트 양식, 알림 문구가 서로 다름. 산업군마다 서비스를 새로 만들면 같은 배송 로직이 계속 복제됨.

해결 등록 → 출고 → 배송 중 → 도착 → 리포트 · 알림 흐름은 하나로 두고, 산업군마다 다른 규칙(도착 판정 · 이상 판정 범위 · 대표 측정값 · 리포트 · 알림 문구 · 입력 검증)만 산업군 값 하나로 고르는 규칙 묶음으로 분리. 자사 디바이스와 외부 실시간 디바이스도 공통 측정값 형식으로 바꿔 같은 흐름에 넣음.

결과 MVP 이후 배차 관제, 제약사 GMP 검증 플랫폼, 해상 고가 화물 플랫폼을 같은 코드베이스에 차례로 출시 (2023.03 – 2023.08).

레거시 무중단 이관 · 고객사 DB 연동

2023 Python Lambda · API Gateway · EventBridge · RDS Proxy · Aurora

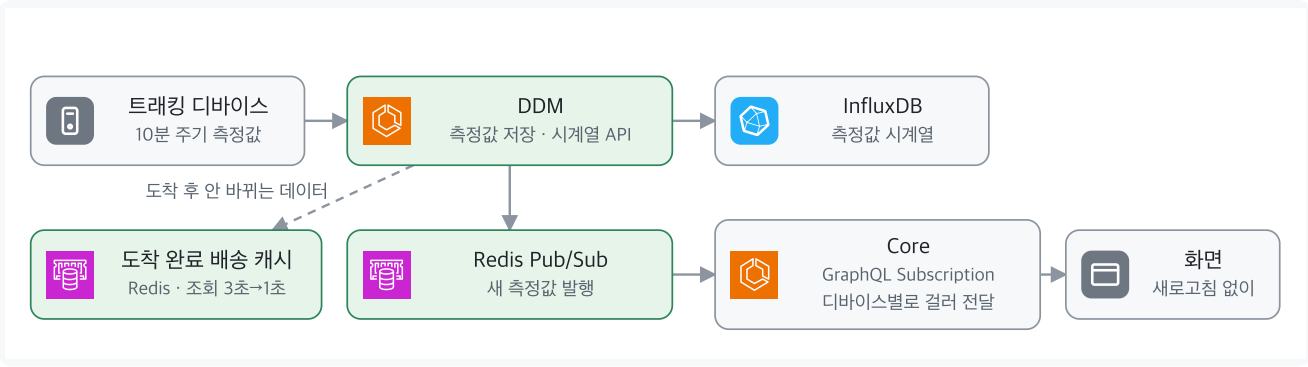


이관 운영 중인 고객사 서비스를 멈추지 않도록 테크 리드와 협의해, 데이터는 Python Lambda로 영업시간에 Cron으로 나눠 사람이 지켜보며 옮기고, 고객사가 쓰던 API는 요청 · 응답 형식을 그대로 맞춰 API Gateway + Lambda로 다시 구축 — 고객사는 코드를 바꾸지 않고 전환. 배송 약 **200만 건** · 디바이스 시계열 약 **5억 건** 이관.

연동 고객사가 중계 DB(MSSQL · Oracle 등)를 두는 방식은 고객사마다 주기 · 구조가 달라 고객사별 Lambda와 EventBridge 스케줄로 구성. 처음엔 Node.js였지만 DB 드라이버가 다양하고 계약 전 POC에서 가장 먼저 붙여야 하는 작업이라 Python으로 통일. VPC 안 Aurora는 RDS Proxy로 접속하고, 쓰기는 작은 트랜잭션 단위로 나눠 성공분은 커밋하고 실패분만 우리 쪽과 고객사에 알림.

실시간 측정값 · 조회 성능

InfluxDB · Redis Pub/Sub · GraphQL Subscription



시계열	배송 테이블에 콤마로 이어 붙인 문자열로 저장하던 측정값(온도 · 습도 · 위치 · 측정 시간)을 InfluxDB로 분리. 기존 TypeORM처럼 쓰는 조회 모듈을 만들어 서비스 코드 변경을 최소화.
실시간	DDM이 새 측정값을 Redis Pub/Sub으로 발행하고, Core의 GraphQL Subscription이 디바이스별로 걸러 화면에 새로고침 없이 전달.
캐시	배송 기간이 길어지면서 시계열 조회가 느려짐 → 도착한 배송은 데이터가 더 바뀌지 않는다는 점을 이용해 Redis에 캐시, 조회 약 **3초 → 1초**.

문제 해결 · 협업

고객사 연동 API 지연

고객사가 10분마다 1만 건 이상 보내는 배송 데이터를 저장 전에 1건씩 라벨 코드 포함 검색(LIKE)으로 중복 검사해 slow query 발생(CloudWatch에서 확인) → 라벨 코드를 100개씩 묶어 한 번에 조회해 쿼리 수를 1/100로 줄이고(1만 건 기준 1만 회 → 100회), 조회 기간은 1개월에서 4일로 좁힘. 고객사 · 배송 정보는 메모리(Record)에 담아 다시 조회하지 않음. 실행 테스트 평균 **20초 → 5초**, 이후 연동 API를 Lambda로 분리.

사내 관리자 서비스(RMS)

NestJS 백엔드를 처음 맡아 설계 · 구현(2022.01 – 03). 디바이스 재고 · 고객사 계정 · 미팅 기록을 한곳에서 관리하고, 관리자가 내부 키로 인증하면 JWT를 발급해 고객사 계정 정보 없이 운영 서비스에 바로 들어가 CS를 처리하게 함.

VOC time

니즈가 정리되지 않은 고객사에 물류 배송 과정을 바탕으로 한 MVP를 먼저 제공하고, 안정화될 때까지 한 달에 한 번 현장 실무자를 만나 기능을 맞춤 개발.